

Node.js + npm — CI/CD Cheat Sheet

Quick reference for DevOps engineers working with Node.js projects

Core Config Files

File	Purpose
<code>package.json</code>	Committed. Defines dependencies and scripts (build/test/start).
<code>package-lock.json</code>	Committed. Pins exact dependency versions for reproducible installs.
<code>~/.npmrc</code>	NOT committed. Private registry URL + auth token — set as a CI secret.

Core CI/CD Commands

Command	Use in CI/CD
<code>npm ci</code>	Clean install from lock file — always use in CI, not <code>npm install</code>
<code>npm test</code>	Runs the test suite — pipeline fails the build on test failure
<code>npm run build</code>	Compiles/bundles (e.g. TypeScript) — skip if plain JS with no build step
<code>docker build -t .</code>	Builds the deployable container image
<code>docker push </code>	Publishes image to registry (ECR/GHCR/Docker Hub) — the CD stage

Typical Pipeline Flow

Checkout → `npm ci` → `npm test` → `npm run build` (if applicable) → `docker build` → `docker push` → deploy

Key point: `node_modules/` is rebuilt fresh from `package-lock.json` on every run — never commit it, never treat it as the artifact. The **Docker image pushed to the registry** is the real, shared deployable artifact that other environments pull from.